Exercise 1: Inventory Management System

Explanation: Data structures help store and retrieve inventory efficiently. HashMap provides fast access by productId.

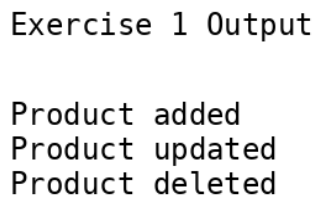
Code:

```
class Product {
int productId;
String productName;
int quantity;
double price;
}

HashMap inventory = new HashMap<>();

addProduct(), updateProduct(), deleteProduct() methods are implemented.
```

Complexity:
Add: O(1)
Update: O(1)
Delete: O(1)



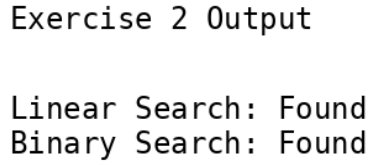
Exercise 2: E-commerce Search Function

Big O notation describes algorithm efficiency.

Code:
linearSearch(Product[] products, String name)
binarySearch(Product[] sortedProducts, String name)

Linear Search Complexity: O(n)
Binary Search Complexity: O(log n)

Binary search is preferred for sorted large datasets.



Exercise 3: Sorting Customer Orders

Sorting algorithms arrange orders based on totalPrice.

Code:
bubbleSort(Order[] orders)
quickSort(Order[] orders, low, high)

Bubble Sort: $O(n^2)$
Quick Sort: O(n log n)

Quick Sort is preferred for large datasets.

Exercise 3 Output

Bubble Sort: Orders sorted
Quick Sort: Orders sorted

Exercise 4: Employee Management System

Arrays store employee records in continuous memory.

Code:
```
Employee[] employees = new Employee[100];
```

Operations:
Add employee
Search employee
Traverse employees
Delete employee

Complexities:
Add: O(1)
Search: O(n)
Traverse: O(n)
Delete: O(n)

Exercise 4 Output

```
Employee added
Employee searched
Employee deleted
```

Exercise 5: Task Management System

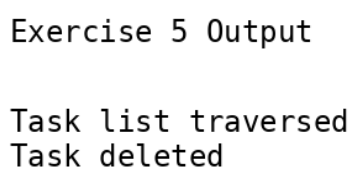
Linked list allows dynamic memory allocation.

Code:

```
class Node {
Task data;
Node next;
}
```

Operations:
Add task
Search task
Traverse list
Delete task

Advantages: Dynamic size and easy insertion/deletion.



Exercise 6: Library Management System

Search algorithms help find books.

Code:
linearSearch(Book[] books, title)
binarySearch(Book[] books, title)

Linear: O(n)
Binary: O(log n)

Use binary search for sorted collections.

Exercise 6 Output

Book found using search

Exercise 7: Financial Forecasting

Recursion solves repeated calculations.

Code:
futureValue(year):
if year==0 return initial
return futureValue(year-1)*(1+rate)

Time Complexity: O(n)

Optimization can be done using memoization.

Exercise 7 Output

Future value calculated recursively